

MINI PROVAS

Documentação Técnica do Sistema

Versão 7.1

Sistema web de aplicação de provas online com autenticação, banco de dados em nuvem e correção automática.

1. Visão Geral

O Mini Provas é um sistema web desenvolvido em Python com FastAPI no back-end e HTML/JavaScript puro no front-end. Permite que usuários se cadastrem, façam login e realizem provas cronometradas com correção automática. Todos os dados ficam na nuvem usando o Supabase como banco de dados.

Tecnologias: Python 3.13 · FastAPI · Uvicorn · Jinja2 · Supabase · HTML · CSS · JavaScript

2. Estrutura de Arquivos

Abaixo está a organização completa do projeto e a função de cada arquivo:

Arquivos raiz

Arquivo	Função
<code>main.py</code>	Ponto de entrada da aplicação. Inicializa o FastAPI, registra as rotas e monta as pastas de estáticos e templates.
<code>run.py</code>	Script de inicialização automática. Instala dependências e inicia o servidor sem comandos manuais.

<code>start.bat</code>	Atalho Windows. Clique duplo para iniciar sem abrir o terminal.
<code>start.sh</code>	Atalho Linux/macOS. Execute para iniciar sem digitar comandos.
<code>.env</code>	Credenciais do Supabase (URL, chave de API). Não deve ser compartilhado publicamente.
<code>requirements.txt</code>	Lista de dependências Python necessárias para o projeto funcionar.

Pasta api/

Arquivo	Função
<code>api/prova_api.py</code>	Rotas HTTP da aplicação: página inicial, listar provas, carregar prova, iniciar e finalizar tentativa. Valida o token JWT antes de operações protegidas.
<code>api/auth_api.py</code>	Rotas de autenticação: <code>/auth/login</code> e <code>/auth/cadastrar</code> via Supabase Auth.

Pasta database/

Arquivo	Função
<code>database/connection.py</code>	Cria a conexão com o Supabase lendo as variáveis do <code>.env</code> . Inicializa o cliente usado por toda a aplicação.

Pasta models/

Arquivo	Função
<code>models/prova.py</code>	Define os modelos de dados: Prova, Questao, Alternativa, Tentativa, Resposta e EntregaProva. Usa Pydantic para validação automática.

Pasta providers/

Arquivo	Função
<code>providers/prova_provider.py</code>	Classe abstrata (interface) que define o contrato de acesso a dados. Garante que o sistema possa trocar de banco sem alterar o restante do código.

<code>providers/supabase_provider.py</code>	Implementação concreta para o Supabase. Contém todas as consultas: provas, questões, alternativas, tentativas e respostas.
<code>providers/provider_factory.py</code>	Factory que instancia o provider correto (atualmente SupabaseProvider).

Pasta repositories/

Arquivo	Função
<code>repositories/prova_repository.py</code>	Intermediário entre o Service e o Provider. O Service nunca acessa o banco diretamente.

Pasta services/

Arquivo	Função
<code>services/prova_service.py</code>	Regras de negócio: busca provas com questões e alternativas, cria tentativas, corrige respostas server-side e calcula nota e aprovação.

Pasta templates/

Arquivo	Função
<code>templates/index.html</code>	Página da prova: título, cronômetro, questões e botão finalizar. Redireciona para login se não autenticado.
<code>templates/login.html</code>	Tela de acesso: abas para login (email + senha) e cadastro (nome + email + senha).

Pasta static/

Arquivo	Função
<code>static/css/style.css</code>	Estilização completa: layout, prova, login, botões, campos e mensagens.
<code>static/js/app.js</code>	Lógica da prova no navegador: carrega questões, registra respostas, controla cronômetro e envia ao servidor.
<code>static/js/auth.js</code>	Lógica de login e cadastro: faz requisições para /auth/* e armazena o token JWT no navegador.

3. Como Iniciar o Sistema

Pré-requisito

Python instalado no computador. Se não tiver, baixe em python.org/downloads e marque "Add Python to PATH" durante a instalação.

Windows

- Extraia o ZIP em qualquer pasta.
- Clique duas vezes em start.bat.
- Aguarde a mensagem "Application startup complete" no terminal.
- Acesse no navegador: <http://127.0.0.1:8000>

Linux / macOS

- Extraia o ZIP e abra o terminal na pasta.
- Execute:

```
bash start.sh
```

- Acesse: <http://127.0.0.1:8000>

Na primeira execução as dependências são instaladas automaticamente. A partir da segunda vez o sistema inicia imediatamente.

4. Fluxo do Sistema

- 1. Cadastro:** O usuário preenche nome, email e senha. A conta é criada via Supabase Auth.
- 2. Login:** Email e senha são validados no Supabase. O token JWT é armazenado no navegador.
- 3. Carregamento:** O front-end busca a primeira prova disponível com questões e alternativas.
- 4. Tentativa:** Ao carregar a prova, o sistema registra automaticamente uma tentativa vinculada ao usuário logado.
- 5. Cronômetro:** O tempo limite da prova é iniciado. Ao zerar, a prova é finalizada automaticamente.
- 6. Correção:** O servidor verifica cada alternativa no banco, calcula a nota e retorna o resultado.
- 7. Resultado:** O sistema exibe acertos, nota final e se o aluno foi aprovado (nota ≥ 6).

5. Banco de Dados (Supabase)

Cinco tabelas principais no Supabase:

Tabela	Descrição
<code>provas</code>	Título, descrição, tempo limite e nota máxima de cada prova.
<code>questoes</code>	Cada questão pertence a uma prova: enunciado, ordem e peso.
<code>alternativas</code>	Cada alternativa pertence a uma questão e indica se é a correta.
<code>tentativas</code>	Registra cada vez que um usuário inicia uma prova, com nota e status final.
<code>respostas</code>	Alternativa escolhida em cada questão e se o aluno acertou.

A autenticação é gerenciada pelo Supabase Auth (`auth.users`) — sem necessidade de tabela de usuários manual.

6. Segurança

- Rotas `/iniciar` e `/finalizar` exigem token JWT válido em todas as requisições.
- O token é gerado pelo Supabase Auth no login e armazenado no navegador.
- O servidor valida o token a cada requisição — o ID do usuário vem do token, nunca do front-end.
- Row Level Security (RLS) ativa no Supabase: cada usuário acessa apenas seus próprios dados.
- A correção das respostas é feita no servidor, consultando o banco — o front-end não decide quem acertou.

7. Histórico de Versões

O projeto passou por diversas iterações. A seguir o histórico completo das versões identificadas a partir dos arquivos entregues:

Versão	Arquivo	Principais mudanças
Protótipo	<code>mini_provas.zip</code>	Versão inicial apenas com a API, sem banco de dados e sem front-end. Estrutura base com Provider Pattern, rotas <code>/iniciar-prova</code> , <code>/finalizar-prova</code> e <code>/resultado</code> . Armazenamento em

		memória (sem Supabase). Bem documentada e comentada para fins didáticos.
Com front-end	back_end.zip	Integração com Supabase, front-end HTML/JS com cronômetro, templates Jinja2. Versão com 9 bugs: UUID quebrando JS, sem rota /iniciar, resultado exibindo campos inexistentes, SUPABASE_URL apontando para dashboard, main.py sem uvicorn.run, correção de provas feita no front-end (inseguro), entre outros. Comentários pessoais da autora no código revelam a luta com os erros.
Com auth e ranking	projeto_corrigido.zip	Versão avançada com autenticação própria (PBKDF2 hash de senha), ranking por prova, feedback de IA simulada (IAProviderSimulado) e preparação para IA externa. Front-end com 4 telas: login, prova, resultado e ranking. Cronômetro com anel SVG animado.
v4 — Bugs corrigidos	mini_provas_corrigido.zip	Correção dos 9 bugs da versão back_end.zip. Adicionados scripts start.bat, start.sh e run.py para inicialização sem terminal. SUPABASE_URL corrigida. Correção de provas migrada para o servidor. UUID tratado corretamente no front-end.
v7.1 — Versão atual	(versão final)	Autenticação completa via Supabase Auth com JWT. RLS por usuário ativo no banco (cada usuário acessa apenas seus dados). Rota /auth/login e /auth/cadastrar. Token enviado em todas as requisições protegidas. Logout disponível na tela da prova e no resultado.

8. Bugs Corrigidos (v4)

Ao migrar de back_end.zip para a versão corrigida, os seguintes problemas foram identificados e resolvidos:

#	Arquivo	Problema → Correção
1	.env	SUPABASE_URL apontava para o dashboard → corrigida para a URL da API (*.supabase.co).
2	main.py	Sem if __name__ == '__main__' → adicionado uvicorn.run() para execução direta.
3	api/prova_api.py	Rota /iniciar inexistente no router → adicionada POST /iniciar.

4	<code>providers/prova_provider.py</code>	Método abstrato <code>buscar_alternativa</code> faltando → adicionado ao contrato.
5	<code>providers/supabase_provider.py</code>	Implementação de <code>buscar_alternativa</code> ausente → adicionada.
6	<code>repositories/prova_repository.py</code>	Método <code>buscar_alternativa</code> faltando → adicionado.
7	<code>services/prova_service.py</code>	acertou vinha do front-end (inseguro) e <code>iniciar_tentativa</code> retornava lista inteira → correção server-side e extração do item correto.
8	<code>static/js/app.js</code>	UUID com hífen colocado cru no HTML quebrava o JavaScript → substituído por <code>addEventListener</code> .
9	<code>static/js/app.js</code>	Resultado exibia <code>resultado.tempo</code> e <code>resultado.dentro_do_tempo</code> (campos inexistentes) → corrigido para acertos, nota e aprovado.